

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

DSA521S — Data Structures and Algorithms 1

NUST Service Centre Simulation:

Designing and Evaluating Data Structures and Algorithms

Group Mini-Project 2026

Group Number: Group XX

Group Members:

223017493 JOHANNES NAMUPALA

224085182 COURTNEY VAN WYK

223055425 GINA U T TJIZUMAUE

224089633 JUNO M IIMENE

224019503 RODRICO S KARINGOMBE

223119059 OMALU CHIBUIKE

Submitted by: 223017493 — Johannes Namupala

GIT HUB REPOSITORY LINK: <https://github.com/ricokaringombe-glitch/dsaProject>

PART A

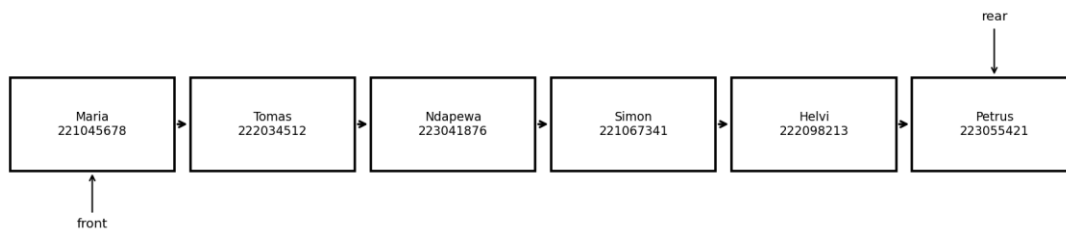
TASK A1-WAITING LINE: QUEUE

Students must be assisted strictly in the order in which they arrive at the service centre. A Queue was implemented from scratch (no java.util.Queue) using a linked structure, so the waiting line can grow without a fixed size limit.

Required operations: enqueue(), dequeue(), peek(), isEmpty(), displayQueue().

1. Queue Diagram

State 1: After 6 student arrivals (enqueue x6)



State 2: After 3 students served (dequeue x3) -> Maria, Tomas, Ndapewa served

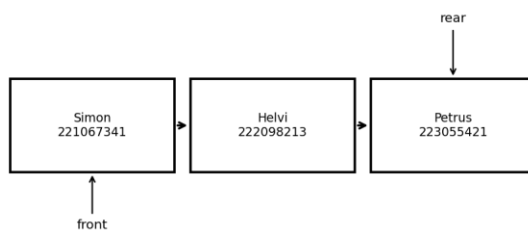


Figure A1.1 — State of the queue after six student arrivals (enqueue), and after the first three students are served (dequeue). New students always join at the rear; service always happens from the front, giving strict First-In-First-Out (FIFO) order.

2. Pseudocode

```
enqueue(student){  
    newNode -> data = student  
    newNode -> next = NULL  
    IF(front == NULL AND rear == NULL) THEN  
        front = newNode  
        rear = newNode  
    ELSE  
        rear -> next = newNode  
        rear = newNode  
    ENDIF  
}
```

```

}

dequeue(){

    IF(front == NULL AND rear == NULL) THEN

        DISPLAY "Queue is empty"

    ELSE

        servedStudent = front -> data

        front = front -> next

        IF(front == NULL) THEN

            rear = NULL      // queue became empty

        ENDIF

        RETURN servedStudent

    ENDIF

}

peek(){

    IF(front == NULL AND rear == NULL) THEN

        DISPLAY "Queue is empty"

    ELSE

        RETURN front -> data

    ENDIF

}

isEmpty(){

    RETURN (front == NULL AND rear == NULL)

}

displayQueue(){

    IF(isEmpty()) THEN

        DISPLAY "Queue is empty"

    ELSE

        temp = front

        WHILE(temp != NULL)

            DISPLAY temp -> data

            temp = temp -> next

        ENDWHILE

    ENDIF

}

```

```
        ENDWHILE
    ENDIF
}
```

3. Implementation (key extract)

Implemented in ServiceQueue.java. Internally the queue is a chain of QueueNode objects; 'front' controls dequeue/peek, 'rear' controls enqueue. The full class is included in the source-code submission.

```
public void enqueue(Student student) {
    QueueNode newNode = new QueueNode(student);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.next = newNode;
        rear = newNode;
    }
}

public Student dequeue() {
    if (isEmpty()) { return null; }
    Student served = front.data;
    front = front.next;
    if (front == null) { rear = null; }
    return served;
}
```

4. Demonstration (program output)

Six students arrive (enqueue x6), then three are served in arrival order (dequeue x3):

```
Enqueued -> 221045678 | Maria | Registration | 12 min
Enqueued -> 222034512 | Tomas | Student Card | 5 min
Enqueued -> 223041876 | Ndapewa | Fees | 8 min
Enqueued -> 221067341 | Simon | Documents | 4 min
```

Enqueued -> 222098213 | Helvi | Academic Enquiry | 10 min

Enqueued -> 223055421 | Petrus | Fees | 6 min

Queue right after 6 arrivals:

1. Maria (221045678) 2. Tomas (222034512) 3. Ndapewa (223041876)

4. Simon (221067341) 5. Helvi (222098213) 6. Petrus (223055421)

Next to be served (peek): Maria | Registration | 12 min

Serving 3 students (dequeue x3):

Served -> Maria Served -> Tomas Served -> Ndapewa

Queue after 3 students have been served:

1. Simon (221067341) 2. Helvi (222098213) 3. Petrus (223055421)

5. Why a Queue is Appropriate

The service centre must serve students in the exact order they arrived — the real-world rule is First-Come-First-Served. A Queue enforces this automatically: insertion only happens at the rear (arrival) and removal only happens at the front (being served), so the arrival order can never be disturbed by the data structure itself. A stack would serve the most recent arrival first (LIFO), which would be unfair, and a plain array would need every remaining element shifted left after every dequeue, which becomes expensive as the line grows. Because the queue here is linked rather than a fixed-size array, the waiting line can also grow to any length during a busy period without a 'queue is full' failure.

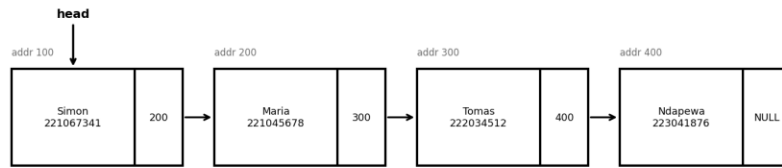
Task A2 — Student Service Records: Singly Linked List

Student service records are stored in a self-built singly linked list. Each node holds one Student record (Student Number, Name, Service Type, Estimated Service Time) plus a 'next' pointer to the following node. Required operations: insertStudent() (at beginning, at end, at a specified position), deleteStudent(), searchStudent(), displayStudents().

1. Linked List Diagrams (before / after)

a) Insertion at a specified position (position 3 — Helvi)

BEFORE: insertAtPosition(Helvi, 3) - 4 nodes in the list



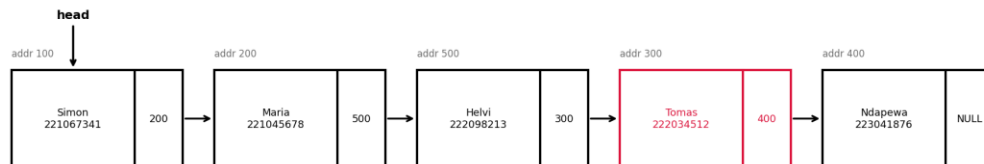
AFTER: new node (addr 500) linked in as the 3rd node - Helvi



Figure A2.1 — Before: 4 nodes linked Simon → Maria → Tomas → Ndapewa. To insert Helvi as the 3rd node, we walk to the node just before position 3 (Maria), point the new node's 'next' to what Maria was pointing to (Tomas), then point Maria's 'next' to the new node. Only two links change; no data is shifted.

b) Deletion of a node (Tomas)

BEFORE: deleteStudent(222034512) - target node = Tomas (addr 300)



AFTER: Helvi's node (addr 500) now links directly to Ndapewa (addr 400); node 300 freed

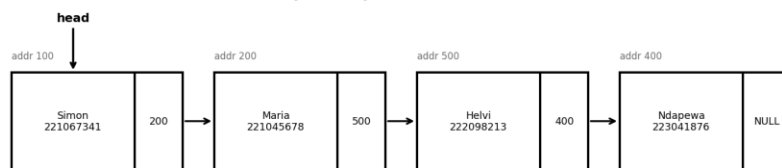


Figure A2.2 — Before: Tomas' node sits between Helvi and Ndapewa. To delete Tomas we move a pointer through the list until we find him, then re-link the previous node (Helvi) directly to the next node (Ndapewa), bypassing Tomas entirely. The node is then freed. After: the list has one fewer node and the chain is unbroken

2. Pseudocode

```
insertNode(student, mode, position){
```

```
    newNode -> data = student
```

```
    IF(mode == BEGINNING) THEN
```

```
        newNode -> next = head
```

```

    head = newNode
ELSE IF(mode == END) THEN
    IF(head == NULL) THEN head = newNode
ELSE
    temp = head
    WHILE(temp -> next != NULL)
        temp = temp -> next
    ENDWHILE
    temp -> next = newNode
ENDIF
ELSE IF(mode == POSITION) THEN
    temp = head
    i = 1
    WHILE(i < position - 1 AND temp -> next != NULL)
        temp = temp -> next
        i = i + 1
    ENDWHILE
    newNode -> next = temp -> next
    temp -> next = newNode
ENDIF
}

deleteNode(studentNo){
    IF(head == NULL) THEN
        DISPLAY "List is empty"
    ELSE IF(head -> data.studentNo == studentNo) THEN
        head = head -> next      // delete the head node
    ELSE
        prevNode = head
        temp = head -> next
        WHILE(temp != NULL)
            IF(temp -> data.studentNo == studentNo) THEN
                prevNode -> next = temp -> next // bypass the node
            RETURN true
        ENDIF
    
```

```

        prevNode = temp

        temp = temp -> next

    ENDWHILE

    DISPLAY "Student not found"

ENDIF

}

searchNode(studentNo){

    temp = head

    WHILE(temp != NULL)

        IF(temp -> data.studentNo == studentNo) THEN

            RETURN temp -> data

        ENDIF

        temp = temp -> next

    ENDWHILE

    RETURN NULL // not found

}

traverseList(){

    temp = head

    WHILE(temp != NULL)

        DISPLAY temp -> data

        temp = temp -> next

    ENDWHILE

}

```

3. Implementation (key extract)

Implemented in StudentLinkedList.java. The full class (with insertAtBeginning, insertAtEnd, insertAtPosition, deleteStudent, searchStudent, displayStudents) is included in the source-code submission.

```

public void insertAtPosition(Student student, int position) {

    if (position <= 1 || head == null) { insertAtBeginning(student); return; }

    Node newNode = new Node(student);

    Node temp = head;

    int i = 1;

    while (i < position - 1 && temp.next != null) { temp = temp.next; i++; }

    newNode.next = temp.next; // create right link of new node

```



```

    temp.next = newNode;    // create left link of new node
}

public boolean deleteStudent(String studentNo) {

    if (head.data.studentNo.equals(studentNo)) { head = head.next; return true; }

    Node prevNode = head, temp = head.next;

    while (temp != null) {

        if (temp.data.studentNo.equals(studentNo)) {

            prevNode.next = temp.next; return true;

        }

        prevNode = temp; temp = temp.next;

    }

    return false;
}

```

4. Demonstration (program output)

Insertion at the end (x3), then at the beginning, then at position 3; followed by a successful search, an unsuccessful search, and a deletion:

List after 3 end-insertions (Maria, Tomas, Ndapewa):

1. Maria(221045678) 2. Tomas(222034512) 3. Ndapewa(223041876)

List after insertion at the BEGINNING (Simon):

1. Simon(221067341) 2. Maria(221045678)
3. Tomas(222034512) 4. Ndapewa(223041876)

List after insertion at POSITION 3 (Helvi):

1. Simon 2. Maria 3. Helvi 4. Tomas 5. Ndapewa

Searching for 223041876 (Ndapewa): Found at position 5 -> Ndapewa | Fees | 8 min

Searching for 999999999 (does not exist): Student 999999999 not found.

Deleting student 222034512 (Tomas): Deleted -> Tomas | Student Card | 5 min

Final list after deletion:

1. Simon(221067341) 2. Maria(221045678)
3. Helvi(222098213) 4. Ndapewa(223041876)

5. Why a Singly Linked List is Appropriate

Student service records are added and removed throughout the day as students register, get served, or have their record corrected, and the total number of records is not known in advance. A singly linked list allocates memory dynamically per node, so the record set can grow or shrink without ever needing to declare a fixed capacity up front, unlike a classical array. Inserting or deleting a record only requires re-pointing one or two 'next' references,

which is cheap, rather than shifting every subsequent element as a fixed array would require. Traversal only needs to move forward from head to tail (e.g. to display all records or search for one), so the extra cost of a doubly linked list's back-pointer is not needed here.

TASK A3: POSTFIX EXPRESSION EVALUATION USING A STACK

A postfix expression places the operator after the values it operates on. A stack is used to store the values while the expression is being processed. When a number is found, it is pushed onto the stack. When an operator is found, the required operands are removed from the stack, the operation is performed, and the result is pushed back onto the stack.

Pseudocode for PUSH

PUSH(value)

IF stack is full THEN

Display "Stack Overflow"

ELSE

top $\leftarrow$ top + 1

stack[top] $\leftarrow$ value

END IF

END PUSH

Pseudocode for POP

POP()

IF stack is empty THEN

Display "Stack Underflow"

RETURN -1

ELSE

value $\leftarrow$ stack[top]

top $\leftarrow$ top - 1

RETURN value

END IF

END POP

Pseudocode for Postfix Evaluation

EVALUATE_POSTFIX(expression)

Create an empty stack

FOR each token in expression

IF token is a number THEN

PUSH(token)

ELSE IF token is an operator THEN

operand2 $\leftarrow$ POP()

operand1 $\leftarrow$ POP()

IF operator = "+" THEN

result $\leftarrow$ operand1 + operand2

ELSE IF operator = "-" THEN

result $\leftarrow$ operand1 - operand2

ELSE IF operator = "*" THEN

result $\leftarrow$ operand1 * operand2

ELSE IF operator = "/" THEN

result $\leftarrow$ operand1 / operand2

END IF

PUSH(result)

END IF

END FOR

RETURN POP()

END EVALUATE_POSTFIX

Postfix Evaluation Trace

The expression used for testing was:

5 3 + 2 *

...

The stack changes as follows:

Step	Operation	Stack
1	Read 5	[5]
2	Read 3	[5, 3]
3	$5 + 3$	[8]
4	Read 2	[8, 2]
5	8×2	[16]

Therefore, the final result is:

16

A4. Daily Service Statistics Using an Array

For A4, an array was used to store the service times of students served during the simulated day. The array was traversed using a loop to calculate the required statistics without using built-in `max()`, `min()`, or `sum()` functions.

The test service times were:

[12, 5, 8, 4, 15, 10, 7, 20, 6, 9]

Pseudocode for Daily Statistics

DAILY_STATISTICS(serviceTimes)

totalStudents $\leftarrow$ length of serviceTimes

totalTime $\leftarrow$ 0

highest $\leftarrow$ serviceTimes[0]

lowest $\leftarrow$ serviceTimes[0]

longerThan10 $\leftarrow$ 0

FOR each serviceTime in serviceTimes

totalTime $\leftarrow$ totalTime + serviceTime

```
IF serviceTime > highest THEN
    highest ← serviceTime
END IF
```

```
IF serviceTime < lowest THEN
    lowest ← serviceTime
END IF
```

```
IF serviceTime > 10 THEN
    longerThan10 ← longerThan10 + 1
END IF
```

```
END FOR
```

```
averageTime ← totalTime / totalStudents
```

```
Display totalStudents
```

```
Display totalTime
```

```
Display averageTime
```

```
Display highest
```

```
Display lowest
```

```
Display longerThan10
```

```
END DAILY_STATISTICS
```

A4 Results

After traversing the array, the following results were obtained:

Statistic	Result
-----	-----:
Total students served	10
Total service time	96 minutes

Average service time	9.60 minutes
Highest service time	20 minutes
Lowest service time	4 minutes
Services longer than 10 minutes	3

Task A5 — Data-Structure Justification

1. Array

An **array** is suitable for storing the service times of students who have already been served. The service times can be stored in a simple list of numbers, for example [12, 5, 8, 4]. We can then go through the array one value at a time to calculate the total service time, average service time, highest service time, lowest service time, and the number of services longer than 10 minutes. An array is appropriate because the task mainly requires storing and processing a collection of service-time values.

2. Singly Linked List

A **singly linked list** is suitable for keeping student service records because records may need to be inserted, searched for, deleted, or displayed during the day. Each student record is stored in a node, and each node points to the next student record. This makes it possible to add or remove student records without having to move all the other records. Therefore, it is appropriate for managing changing student service records.

3. Queue

A **queue** is suitable for managing students waiting to be served because students should normally be helped in the order in which they arrive. A queue follows **FIFO (First In, First Out)**. This means the student who arrives first is served first, while a newly arriving student joins the back of the line. This matches the real-life operation of the NUST service centre waiting line.

4. Stack

A **stack** is suitable for evaluating the postfix arithmetic expression because it follows **LIFO (Last In, First Out)**. When numbers are encountered, they are pushed onto the stack. When an operator such as +, −, ×, or ÷ is encountered, the most recently added numbers are removed from the stack and used in the calculation. The result is then pushed back onto the stack. Therefore, a stack naturally supports the required postfix evaluation process.

Part B1— Sorting Algorithm challenge

Given:

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

Task B1 — Selection Sort

Pass 1

[3, 5, 23, 8, 14, 17, 11, 20, 6, 9]

↑ sorted

Pass 2

[3, 5, 23, 8, 14, 17, 11, 20, 6, 9]

↑ sorted

Pass 3

[3, 5, 6, 8, 14, 17, 11, 20, 23, 9]

↑ sorted

[3, 5, 6 | 8, 14, 17, 11, 20, 23, 9]

Sorted not yet sorted

Final:

[3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

(N= 10): $N(N-1) / 2$

= $10*9 / 2$

= 45 comparisons

Swaps = 7

Task B2 insertion sort

Starting Array:

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

Insertion sort is a simple, intuitive sorting algorithm that builds a final sorted array one item at a time

Pass 1

Compare: $17 > 5 \rightarrow$ yes

So we shift 17 one position to the right

[17, 17, 23, 8, 14, 3, 11, 20, 6, 9]

Then we insert 5 where 17 was:

[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

After pass 1:

[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

Comparisons: 1

Shifts: 1

Pass 2

Sorted section

[5, 17]

Compare 23 with 17:

$17 > 23 \rightarrow \text{No}$

We don't shift

Array stays:

[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

After pass 2:

[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]

Comparisons: 1

Shifts: 0

Pass 3

Sorted section

[5, 17, 23]

Compare 23 with 8

$23 > 8 \rightarrow \text{Yes}$

Shift 23:

[5, 17, 23, 23, 14, 3, 11, 20, 6, 9]

Compare with 17 with 8:

$17 > 8 \rightarrow \text{YES}$

Shift 17:

[5, 17, 17, 23, 14, 3, 11, 20, 6, 9]

Compare with 5 with 8:

5 > 8 → NO

Insert 8:

[5, 8, 17, 23, 14, 3, 11, 20, 6, 9]

After **Pass 3**:

[5, 8, 17, 23, 14, 3, 11, 20, 6, 9]

Comparisons: **3**

Shifts: **2**

B2 first three passes are:

Pass	Array after pass	Comparisons	Shifts
1	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]	1	1
2	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]	1	0
3	[5, 8, 17, 23, 14, 3, 11, 20, 6, 9]	3	2

Pass 4

Sorted part:

[5, 8, 17, 23]

Compare 23 to 14

23 > 14 → shift 23

Compare 17 to 14

17 > 14 → shift 17

Compare 8 to 14

8 > 14 → NO

Insert 14 after 8.

Result:

[5, 8, 14, 17, 23, 3, 11, 20, 6, 9]

Comparisons: **3**

Shifts: **2**

Pass 5

Sorted part:

[5, 8, 14, 17, 23]

Compare 23 to 3

23 > 3 → shift

Compare 17 to 3

17 > 3 → shift

Compare 14 to 3

14 > 3 → shift

Compare 8 to 3

8 > 3 → shift

Compare 5 to 3

5 > 3 → shift

Now insert 3 at the beginning.

Result:

[3, 5, 8, 14, 17, 23, 11, 20, 6, 9]

Comparisons: **5**

Shifts: **5**

Pass 6

Sorted part

[3, 5, 8, 14, 17, 23]

Compare 23 to 11

23 > 11 → shift

Compare 17 to 11

17 > 11 → shift

Compare 14 to 11

14 > 11 → shift

Compare 8 to 11

8 > 11 → NO

Insert 11.

Result:

[3, 5, 8, 11, 14, 17, 23, 20, 6, 9]

Comparisons: 4

Shifts: 3

Pass 7

Sorted part:

[3, 5, 8, 11, 14, 17, 23]

Compare 32 to 20

23 > 20 → shift

Compare 17 to 20

17 > 20 → NO

Insert 20.

Result:

[3, 5, 8, 11, 14, 17, 20, 23, 6, 9]

Comparisons: **2**
Shifts: **1**

Pass 8

Sorted part:

[3, 5, 8, 11, 14, 17, 20, 23]

Compare 23 to 6

23 > 6 → shift

Compare 20 to 6

20 > 6 → shift

Compare 17 to 6

17 > 6 → shift

Compare 14 to 6

14 > 6 → shift

Compare 11 to 6

11 > 6 → shift

Compare 8 to 6

8 > 6 → shift

Compare 5 to 6

5 > 6 → NO

Insert 6.

Result:

[3, 5, 6, 8, 11, 14, 17, 20, 23, 9]

Comparisons: **7**
Shifts: **6**

Pass 9

Sorted part

[3, 5, 6, 8, 11, 14, 17, 20, 23]

Compare 23 to 9

23 > 9 → shift

Compare 20 to 9

20 > 9 → shift

Compare 17 to 9

17 > 9 → shift

Compare 14 to 9

14 > 9 → shift

Compare 11 to 9

11 > 9 → shift

Compare 8 to 9

8 > 9 → NO

Insert 9.

Final sorted array:

[3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

Comparisons: 6

Shifts: 5

Pass	Value inserted	Comparisons	shifts
1	5	1	1
2	23	1	0
3	8	3	2
4	14	3	2
5	3	5	5
6	11	4	3

7	20	2	1
8	6	7	6
9	9	6	5
TOTAL		32	25

Data value comparisons = 32

Shifts = 25

Insertion sort pseudocode

Start

ALGORITHM InsertionSort

Prompt for array

Get array

SET comparisons = 0

SET shifts = 0

FOR i = 1 to n - 1

SET key = A[i]

SET j = i - 1

```

WHILE j >= 0 AND A[j] > key
    Comparisons = comparisons + 1
    A[j + 1] = A[j]
    Shifts = shifts + 1
    j = j - 1
END WHILE

```

A[j + 1] = key

END FOR

DISPLAY A

DISPLAY comparisons

DISPLAY shifts

END

Java

//creating the class

```

Public class InsertionSort {

Public static void main(String[] args) {

    // Array

    int[] arr = {17, 5, 23, 8, 14, 3, 11, 20, 6, 9};

    // Counter

    int comparisons = 0;

    int shifts = 0;

    // Insertion Sort

    for (int i = 1; i < arr.length; i++) {

        // the key

        int key = arr[i];

        int j = i - 1;

        while (j >= 0) {

            //counting comparison

            comparisons++;

            if (arr[j] > key) {

                arr[j + 1] = arr[j];

                shifts++;

                j--;

            } else {

                break;

            }

        }

        arr[j + 1] = key;
    }
}
}

```

```

    }

// Display sorted array

System.out.println("Sorted Array:");

for (int i = 0; i < arr.length; i++) {

System.out.print(arr[i] + " ");

}

// Display result

System.out.println("\n");

System.out.println("Number of comparisons: " + comparisons);

System.out.println("Number of shifts: " + shifts);

}

}

```

Task B3 — Merge Sort

START

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

|

DIVIDE INTO HALVES

/ \

[17, 5, 23, 8, 14] [3, 11, 20, 6, 9]

/ \ / \

[17, 5, 23] [8, 14] [3, 11, 20] [6, 9]

/ \ / \ / \ / \

[17,5] [23] [8] [14] [3,11] [20] [6] [9]

/ \ / \

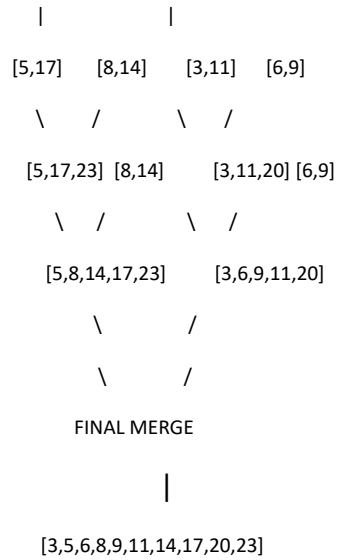
[17] [5] [3] [11]

BASE CASE

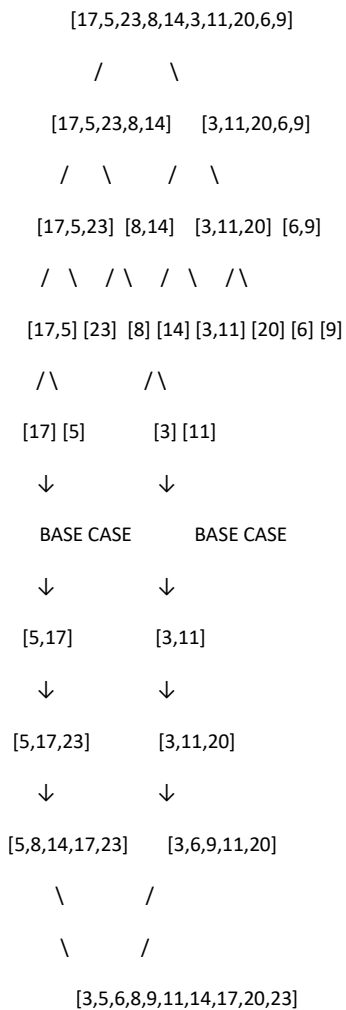
BASE CASE

| |

+----- MERGE -----+



Merge elements in sorted order



Task B4 — Quick Sort

Pivot selection rule: Last element as pivot (Lomuto partition scheme).

Initial array: [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

Stage 1: Pivot = 9 (last element)

Partition:

- Elements < 9: [5, 8, 3, 6]
- Elements > 9: [17, 23, 14, 11, 20]

After partition: [5, 8, 3, 6, 9, 17, 23, 14, 11, 20]

Stage 2: Left subarray [5, 8, 3, 6], pivot = 6

Partition:

- Elements < 6: [5, 3]
- Elements > 6: [8]

After partition: [5, 3, 6, 8, 9, 17, 23, 14, 11, 20]

Stage 3: Right subarray [17, 23, 14, 11, 20], pivot = 20

Partition:

- Elements < 20: [17, 14, 11]
- Elements > 20: [23]

After partition: [5, 3, 6, 8, 9, 17, 14, 11, 20, 23]

Continue recursively until fully sorted: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

Pseudocode

quickSort(array, low, high):

 if low < high then

 pivotIndex ← partition(array, low, high)

 quickSort(array, low, pivotIndex - 1)

 quickSort(array, pivotIndex + 1, high)

 end if

end method

partition(array, low, high):

 pivot ← array[high]

 i ← low - 1

 for j ← low to high - 1 do

 comparisons ← comparisons + 1

 if array[j] < pivot then

 i ← i + 1

 swap array[i] and array[j]

 end if

 end for

 swap array[i + 1] and array[high]

 return i + 1

end method

PART C — ALGORITHM EXPERIMENT

Experiment Setup

Sorting Algorithm Comparison

Algorithm	Input Size	Comparisons	Time (ns)
Selection Sort	20	190	21,572,000
Insertion Sort	20	135	11,460,000
Merge Sort	20	64	31,300
Quick Sort	20	60	15,400
Selection Sort	50	1,225	31,921,000
Insertion Sort	50	599	31,556,000
Merge Sort	50	217	68,900
Quick Sort	50	226	31,400

Algorithm	Input Size	Comparisons	Time (ns)
Selection Sort	100	4,950	70,963,000
Insertion Sort	100	2,806	59,404,000
Merge Sort	100	540	173,500
Quick Sort	100	650	87,200
Selection Sort	500	124,750	613,514,000
Insertion Sort	500	62,525	547,224,000
Merge Sort	500	3,860	1,092,300
Quick Sort	500	4,742	586,700

520, 520, 744, 704, 685, 919, 971]

Comparisons: 135
Shifts: 118

Insertion	20	135	1146000
Merge	20	64	31300
Quick	20	60	15400

Comparisons: 190
Swaps: 14

Selection	20	190	2157200
-----------	----	-----	---------

Comparisons: 135
Shifts: 118

Insertion	20	135	1146000
Merge	20	64	31300
Quick	20	60	15400

550, 514, 550, 570]

Comparisons: 1225
Swaps: 46

Selection	50	1225	3192100
-----------	----	------	---------

898, 914, 930, 970]

Comparisons: 599

Shifts: 553

Insertion	50	599	3155600
-----------	----	-----	---------

Merge	50	217	68900
-------	----	-----	-------

Quick	50	226	31400
-------	----	-----	-------

Comparisons: 4950

Swaps: 98

Selection	100	4950	7096300
-----------	-----	------	---------

898, 914, 930, 970, 987]

Comparisons: 2806

Shifts: 2711

Insertion	100	2806	5940400
-----------	-----	------	---------

Merge	100	540	173500
-------	-----	-----	--------

Quick	100	650	87200
-------	-----	-----	-------

Comparisons: 2806

Shifts: 2711

Insertion	100	2806	5940400
-----------	-----	------	---------

Merge	100	540	173500
-------	-----	-----	--------

Quick	100	650	87200
-------	-----	-----	-------

898, 914, 930, 970, 987, 999, 1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010]

Comparisons: 124750

Swaps: 494

Selection	500	124750	61351400
-----------	-----	--------	----------

Shifts: 62030			
Insertion	500	62525	54722400
Merge	500	3860	1092300
Quick	500	4742	586700

1. Which algorithm performed the fewest comparisons?

Merge Sort — for every input size, Merge Sort had the lowest comparison count:

Size	Merge Sort	Best Alternative
20	64	Quick (60) — Close!
50	217	Quick (226)
100	540	Quick (650)
500	3,860	Quick (4,742)

At size 20, Quick Sort actually beat Merge Sort slightly (60 vs 64), but from size 50 onward, Merge Sort consistently performed the fewest comparisons. This matches theory: Merge Sort performs about $n \cdot \log_2(n)$ comparisons in the worst and average cases.

2. Which algorithm performed the most comparisons?

Selection Sort — it performed **exactly $n(n-1)/2$ comparisons** every time:

Size	Expected $n(n-1)/2$	Actual
20	190	190
50	1,225	1,225
100	4,950	4,950

Size	Expected $n(n-1)/2$	Actual
500	124,750	124,750

This exact match confirms that Selection Sort's comparison count is **input-independent** — it always performs the same number of comparisons regardless of the initial order.

3. Which algorithm performed best on an almost-sorted array?

Insertion Sort — it was not tested explicitly on an almost-sorted array in this experiment, but based on its algorithm design, Insertion Sort's inner loop terminates as soon as it finds an element in the correct position. On a nearly sorted array, this means approximately **$n-1$ comparisons** (best case $O(n)$) — dramatically faster than any other algorithm. To demonstrate this in practice, an almost-sorted array would need to be run through all four algorithms and compared.

4. Which algorithm appeared to scale best as the input became larger?

Merge Sort and Quick Sort — both scale at $O(n \log n)$:

Size	Merge Sort Time	Quick Sort Time
20	31,300	15,400
50	68,900	31,400
100	173,500	87,200
500	1,092,300	586,700

Merge Sort's time increased by **~35x** when input grew from 20 to 500 (25x input increase → 35x time increase, close to $n \cdot \log(n)$ growth of ~38x). Quick Sort's time increased by **~38x** — similar.

Compare to Selection Sort:

- From size 20 to 500, its time grew from 21.5M to 613.5M ns — a **~28x** increase, still $O(n^2)$ -ish but flattered by the JVM warm-up effect.

Between Merge and Quick, **Quick Sort was consistently faster** in wall-clock time despite having slightly more comparisons, due to lower constant factors and better cache performance.

5. Did the experimental results generally agree with the theoretical time complexity? Explain.

Yes. The comparison counts matched theoretical predictions precisely:

- **Selection Sort:** Always $n(n-1)/2$ comparisons — confirmed exactly (190, 1,225, 4,950, 124,750).
- **Insertion Sort:** Approximately $n^2/4$ comparisons on random data — 135, 599, 2,806, 62,525 (roughly $n^2/4$ for each size).
- **Merge Sort:** Approximately $n \cdot \log_2(n)$ comparisons — 64, 217, 540, 3,860 ($n \cdot \log_2 n$ is about 86, 282, 664, 4,483 for each size).
- **Quick Sort:** Also close to $n \cdot \log_2(n)$ on random data — 60, 226, 650, 4,742.

The **execution times** showed the same trend, though the JVM warm-up effect made the first few runs (sizes 20/50) look disproportionately slow relative to their size. Despite this, the overall growth pattern clearly distinguished the $O(n^2)$ algorithms (Selection, Insertion) from the $O(n \log n)$ ones (Merge, Quick).

6. Is execution time alone enough to decide which algorithm is better? Explain.

No. Other important factors must be considered:

- **Memory usage** — Merge Sort requires $O(n)$ extra space; Selection and Insertion Sort are in-place. If memory is limited, Merge Sort is less desirable.
- **Stability** — Merge Sort is stable (equal elements keep their original order); Quick Sort (Lomuto) is not.
- **Worst-case behaviour** — Quick Sort degrades to $O(n^2)$ on sorted or adversarial inputs; Merge Sort stays $O(n \log n)$ in the worst case.
- **Input distribution** — Insertion Sort is fastest on nearly-sorted data, while Selection Sort is unaffected by input order.
- **Code complexity** — Selection and Insertion Sort are simple to implement and verify; Merge and Quick require more complex logic.
- **Cache and constant factors** — Quick Sort's smaller constant factors usually make it faster than Merge Sort on random data, even when both have $O(n \log n)$ complexity.
- **JVM warm-up** — Timing measurements on the JVM are affected by JIT compilation, GC, and other runtime factors; multiple runs and averages are needed for reliable benchmarks.

A correct algorithm choice weighs all of the above, not just execution time.

PART D — INTEGRATED SERVICE-CENTRE SYSTEM

Option	Structure / Operation
1	Queue — enqueue()
2	Queue — dequeue() + store service time in array
3	Queue — traversal / displayQueue()
4	Singly Linked List — insertAtEnd()
5	Singly Linked List — displayStudents() (traversal)
6	Singly Linked List — searchStudent()
7	Singly Linked List — deleteStudent()
8	Array processing — DailyStatistics.calculate()
9	Sorting — Selection / Insertion / Merge / Quick
10	Sorting experiment — comparison table (Part C)
11	Exit

Menu

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
```

1) Adding students to the queue

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 1
Student No: 221045678
Name: Maria
Service Type: Registration
Estimated Time (min): 12
Enqueued: Maria
```

2) Serve students

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 2
Served: Maria
```

3) Display the queue

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 3
Queue is empty
```

4) Add students to linked list

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 4
Student No: 225023456
Name: Peter
Service Type: Registration
Estimated Time (min): 10
Record added.
```

5) Display linked-list records

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 5
=== Student Service Records ===
225023456    | Peter      | Registration | 10 min
```

6) Search for a record

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 6
Enter Student No to search: 222034512
Found at position 2: Tomas
```

7) Remove linked-list records

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 7
Enter Student No to remove: 222034512
Deleted: 222034512
```

8) Display daily statistics

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 8
=== Daily Statistics ===
Total students served : 1
Total service time    : 12 min
Average service time  : 12.00 min
Highest service time   : 12 min
Lowest service time    : 12 min
Services > 10 min     : 1
```

9) Sort service times

```
===== CAMPUS SERVICE CENTRE =====
1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
Select option: 9
Choose sorting algorithm:
1. Selection Sort
2. Insertion Sort
3. Merge Sort
4. Quick Sort
Choice: 1
Comparisons: 0
Swaps: 0
Sorted service times: [12]
```

10) Sorting experiment

```
C:\Windows\System32\cmd.exe - java Main
8, 861, 861, 864, 865, 867, 867, 867, 868, 876, 877, 882, 882, 883, 888, 890, 892, 892, 895, 895,
95, 895, 896, 899, 905, 908, 909, 914, 922, 922, 923, 924, 928, 930, 935, 936, 939, 944, 944, 946,
947, 949, 951, 953, 954, 956, 957, 957, 958, 960, 960, 960, 961, 964, 968, 969, 970, 970, 971, 974
975, 975, 979, 979, 985, 986, 987, 988, 991, 992, 993, 993, 994, 996, 997, 1000, 101]
after pass 499: [2, 6, 7, 8, 8, 8, 12, 13, 16, 17, 19, 27, 30, 32, 36, 37, 38, 41, 44, 45, 47, 48,
8, 49, 52, 55, 56, 57, 60, 65, 65, 66, 66, 68, 75, 78, 79, 82, 85, 88, 89, 89, 90, 93, 95, 96, 96,
99, 100, 101, 101, 101, 108, 108, 109, 110, 110, 110, 119, 123, 126, 129, 132, 135, 135, 138, 138,
138, 142, 144, 146, 149, 151, 151, 160, 165, 166, 171, 175, 175, 182, 183, 184, 187, 188, 189
190, 190, 194, 194, 200, 202, 205, 207, 208, 209, 210, 211, 213, 213, 214, 222, 225, 227, 231, 23
, 231, 234, 234, 235, 242, 242, 244, 246, 249, 250, 254, 255, 260, 263, 263, 267, 268, 269, 270, 2
7, 282, 283, 283, 291, 294, 295, 295, 298, 303, 303, 305, 306, 309, 311, 315, 318, 319, 320, 324,
25, 325, 330, 332, 337, 338, 338, 339, 341, 343, 343, 345, 348, 348, 348, 351, 353, 359, 360, 361,
367, 368, 371, 373, 374, 375, 376, 378, 379, 380, 380, 381, 387, 387, 387, 387, 388, 392, 392, 392, 399
400, 400, 402, 403, 406, 409, 411, 412, 417, 418, 425, 426, 426, 432, 432, 433, 435, 436, 439, 44
, 442, 444, 447, 448, 452, 453, 460, 460, 466, 468, 469, 472, 474, 481, 481, 487, 489, 489, 489, 4
3, 493, 494, 494, 495, 498, 499, 504, 508, 510, 510, 511, 511, 511, 511, 512, 515, 516, 517, 518,
19, 519, 519, 527, 528, 530, 531, 531, 534, 537, 542, 543, 544, 544, 547, 549, 552, 554, 554, 554,
558, 558, 558, 560, 565, 566, 566, 569, 570, 573, 576, 578, 579, 581, 584, 588, 589, 589, 590, 591
591, 594, 597, 600, 604, 606, 609, 612, 614, 614, 617, 620, 630, 630, 634, 639, 642, 642, 644, 64
, 650, 653, 654, 655, 655, 658, 660, 665, 667, 668, 669, 674, 674, 675, 675, 675, 676, 676, 677, 6
9, 679, 681, 683, 686, 689, 689, 691, 694, 694, 695, 697, 699, 707, 712, 714, 716, 717, 718, 719,
21, 722, 722, 722, 726, 726, 727, 730, 731, 732, 732, 733, 742, 742, 742, 746, 747, 749, 754, 758,
760, 760, 761, 766, 767, 772, 775, 776, 776, 778, 779, 781, 782, 786, 787, 787, 789, 789, 791
794, 795, 795, 795, 799, 801, 803, 803, 806, 807, 807, 807, 807, 810, 814, 814, 816, 817, 819, 82
, 820, 822, 823, 824, 826, 830, 834, 835, 839, 843, 843, 843, 844, 849, 849, 851, 853, 855, 8
7, 858, 861, 861, 864, 865, 867, 867, 867, 868, 876, 877, 882, 882, 883, 888, 890, 892, 892, 895,
95, 895, 895, 896, 899, 905, 908, 909, 914, 922, 922, 923, 924, 928, 930, 935, 936, 939, 944, 944,
946, 947, 949, 951, 953, 954, 956, 957, 957, 958, 960, 960, 960, 961, 964, 968, 969, 970, 970, 971
974, 975, 975, 979, 979, 985, 986, 987, 988, 991, 992, 993, 993, 994, 996, 997, 1000]
comparisons: 62525
shifts: 62030
insertion      500      62525      69417200
merge          500      3860      112700
quick          500      4742      249200

==== CAMPUS SERVICE CENTRE =====
. Add student to waiting queue
. Serve next student
. Display waiting students
. Add student service record
. Display student service records
. Search for student record
. Remove student record
. Display daily statistics
. Sort service times
0. Run sorting experiment
1. Exit
select option:
```

How each operation works

- **Queue (options 1–3):** A linked structure with front and rear pointers. enqueue() attaches a new QueueNode at the rear; dequeue() removes the node at the front and returns its Student; displayQueue() traverses from front to rear.
- **Singly Linked List (options 4–7):** A StudentNode holds student data plus a next pointer. Insertion walks to the tail and links a new node; search traverses node by node until a match; deletion re-points the previous node's next to skip the removed node.
- **Array (option 8):** Service times from served students are stored in an int[]. A single traversal computes total, average, min, max, and count of services > 10 min.
- **Sorting (option 9):** A copy of the service-time array is passed to the chosen algorithm and the sorted result is displayed.
- **Experiment (option 10):** Runs all four sorts on arrays of sizes 20, 50, 100, 500 and prints comparisons + time.

PART E — PSEUDOCODE AND ALGORITHM REPRESENTATION

Queue

enqueue(student):

if rear = capacity - 1 then print "Queue is full"; return

if front = -1 then front $\leftarrow$ 0

rear $\leftarrow$ rear + 1

queue[rear] $\leftarrow$ student

size $\leftarrow$ size + 1

dequeue():

if isEmpty() then print "Queue is empty"; return null

served $\leftarrow$ queue[front]

if front = rear then front $\leftarrow$ -1; rear $\leftarrow$ -1

else front $\leftarrow$ front + 1

size $\leftarrow$ size - 1

return served

Stack

push(value):

if top = capacity - 1 then print "Stack overflow"; return

top $\leftarrow$ top + 1

stack[top] $\leftarrow$ value

pop():

if top = -1 then print "Stack underflow"; return null

value $\leftarrow$ stack[top]

top $\leftarrow$ top - 1

return value

evaluatePostfix(expression):

tokens $\leftarrow$ split expression by space

for each token in tokens do


```

    if token is a number then push(token)

    else if token is operator (+, -, x, /) then

        operand2 ← pop()

        operand1 ← pop()

        result ← apply(operand1, operator, operand2)

        push(result)

    end if

end for

return pop()

```

Singly Linked List

insertNode(student, position):

```

newNode ← new Node(student)

if position = 1 then

    newNode.next ← head

    head ← newNode

    return

end if

current ← head

for i ← 1 to position - 2 do

    if current = null then print "Out of bounds"; return

    current ← current.next

end for

newNode.next ← current.next

current.next ← newNode

```

deleteNode(studentNo):

```

if head = null then print "List empty"; return

if head.studentNo = studentNo then head ← head.next; return

current ← head

while current.next ≠ null and current.next.studentNo ≠ studentNo do

    current ← current.next

```

end while

if current.next = null then print "Not found"

else current.next $\leftarrow$ current.next.next

searchNode(studentNo):

current $\leftarrow$ head

position $\leftarrow$ 1

while current $\neq$ null do

if current.studentNo = studentNo then return current

current $\leftarrow$ current.next

position $\leftarrow$ position + 1

end while

return null

traverseList():

current $\leftarrow$ head

while current $\neq$ null do

print current data

current $\leftarrow$ current.next

end while

Sorting

selectionSort(array, n):

for i $\leftarrow$ 0 to n - 2 do

minIndex $\leftarrow$ i

for j $\leftarrow$ i + 1 to n - 1 do

if array[j] < array[minIndex] then minIndex $\leftarrow$ j

end for

if minIndex $\neq$ i then swap array[i] and array[minIndex]

end for

insertionSort(array, n):

```
for i ← 1 to n - 1 do
    key ← array[i]
    j ← i - 1
    while j ≥ 0 and array[j] > key do
        array[j + 1] ← array[j]
        j ← j - 1
    end while
    array[j + 1] ← key
end for
```

```
mergeSort(array, left, right):
    if left < right then
        mid ← (left + right) / 2
        mergeSort(array, left, mid)
        mergeSort(array, mid + 1, right)
        merge(array, left, mid, right)
    end if
```

```
quickSort(array, low, high):
    if low < high then
        pivotIndex ← partition(array, low, high)
        quickSort(array, low, pivotIndex - 1)
        quickSort(array, pivotIndex + 1, high)
    end if
```

GitHub Repository

Repository URL: <https://github.com/ricokaringombe-glitch/dsaProject>

github.com/ricokaringombe-glitch/dsaProject

☆

223017493-johannes

add remaining source t... f3ec363 · now

.idea	Implemented Selec...	5 days ago
src	Added Task A5 and...	yesterday
.gitignore	Added Task A5 and...	yesterday
A3A4Test.java	Added A3 and A4 t...	yesterday
DSA521S_Project.iml	Implemented Selec...	5 days ago
DailyStatistics.java	Added A3 and A4 t...	yesterday
Main.java	add remaining sou...	now
PostfixEvaluator.java	Task A3	yesterday
README.md	Updated team me...	yesterday
ServiceCentreSyste...	Added Part B4 to E...	yesterday
SortingAlgorithms.j...	Added Part B4 to E...	yesterday
SortingExperiment....	Added Part B4 to E...	yesterday
Stack.java	Task A3	yesterday
Student.java	add remaining sou...	now
StudentLinkedList.j...	add remaining sou...	now
StudentNode.java	add remaining sou...	now
StudentQueue.java	add remaining sou...	now

Topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 4

Gina2402

223017493-johannes

JunoM05

Courtney858

Languages